

Níže je “aktuální pravda z kódu” k jednotkám a boji. Klíčové soubory: `server/gameConfig.js` (definice jednotek, nábor, rychlost) a `server/gameService.js` (bojové staty, simulace boje, kořist, dobývání).

****1) Jednotky - ekonomika, nábor, pohyb**** (`server/gameConfig.js:156`, `server/gameConfig.js:308`, `server/gameConfig.js:326`)

- Každá jednotka má: `cost`, požadovanou budovu+level, `baseRecruitDurationSec`, rychlost `speedTilesPerHour`, `populationCost`.
- Nábor 1+ kusů škáluje lineárně s počtem a zrychluje se s levellem příslušné budovy (ne jen s “minimálním” levellem).
- Pohyb armády je vždy podle ****nejpomalejší**** jednotky v armádě (výpočet travel time bere min rychlost ze selection). (`server/gameConfig.js:326`)

Tabulka (počítané hodnoty pro 1 jednotku, aby bylo vidět tempo hry):

```
```tsv
Jednotky - meta (server/gameConfig.js:156 + výpočty server/gameConfig.js:308/326)
id name req cost(w/s/i) costΣ pop speed(t/h) recruit(1u@req) recruit(1u@lvl10) travel(1 tile) travel(10 tiles)
militia Ozbrojenci barracks L1 18/10/8 36 1 18 35s 27s 4m 10s 41m 40s
archer Lucistnici workshop L1 16/8/14 38 1 16 45s 35s 4m 41s 46m 53s
cavalry Jezdci stable L1 22/14/20 56 1 28 56s 44s 2m 41s 26m 47s
scout Zved stable L3 14/9/11 34 1 36 37s 31s 2m 05s 20m 50s
knight Rytir townhall L1 10000/10000/10000 30000 10 42 1m 20s 1m 02s 1m 47s 17m 51s
ram Beranidla workshop L1 30/22/18 70 1 10 1m 13s 57s 7m 30s 75m 00s
caravan Karavany workshop L1 20/12/10 42 1 14 42s 33s 5m 21s 53m 34s
```
```

Nábor - vzorec (zjednodušeně, `server/gameConfig.js:308`):

- `duration = base \* amount \* (1 - levelReduction) \* RECRUIT\_TIME\_MULTIPLIER`
- `levelReduction` roste s levellem budovy a je capnuté na `0.55`.

Pohyb - vzorec (`server/gameConfig.js:326`):

- `durationSec = (distanceTiles / slowestSpeedTilesPerHour) \* 3600 \* ARMY\_TRAVEL\_TIME\_MULTIPLIER`
- minimálně `MIN\_ARMY\_TRAVEL\_DURATION\_SEC`.

****2) Bojové staty jednotek + kořist**** (`server/gameService.js:2487`, `server/gameService.js:2496`)

- Bojové staty jsou ****oddělené**** od ekonomických (nábor) statů.
- Kořist (carry) je na úrovni jednotek přes `UNIT\_LOOT\_CAPACITY`.

```
```tsv
Jednotky - boj + loot (server/gameService.js:2487/2496 + odvozené metriky ze server/gameService.js:2577/2602)
id atk def hp lootCap hpWeight baseAtkPower/u baseDefPower/u durability casualtyScalexR
militia 11 12 18 20 1.061 11.67 12.73 1.010 0.990
archer 9 14 12 16 0.866 7.79 12.12 0.825 1.212
```
```

| | | | | | | | | | |
|---------|-----|-----|-----|-----|-------|-------|--------|-------|-------|
| cavalry | 18 | 10 | 21 | 80 | 1.146 | 20.62 | 11.46 | 1.091 | 0.917 |
| scout | 4 | 4 | 9 | 0 | 0.750 | 3 | 3 | 0.714 | 1.400 |
| knight | 300 | 255 | 240 | 45 | 1.750 | 525 | 446.25 | 1.650 | 0.497 |
| ram | 0 | 0 | 42 | 0 | 1.620 | 0 | 0 | 1.543 | 0.745 |
| caravan | 0 | 0 | 8 | 250 | 0.707 | 0 | 0 | 0.673 | 1.708 |

Co znamenají odvozené sloupce:

- ``hpWeight`` = váha HP v “power” výpočtu (``server/gameService.js:2577``)
``hpWeight` = clamp(sqrt(hp)/4, 0.5, 1.75)``
- ``baseAtkPower/u`` a ``baseDefPower/u`` = “kolik power přidá 1 jednotka” do součtu (bez budov/taktiky).
Důležité: `**ram + caravan mají atk/def = 0**`, takže do power součtu se nepočítají, ale pořád mohou umírat.
- ``durability` + `casualtyScale`×R`` vychází z ``applyCasualties`` (``server/gameService.js:2602``)
``durability` = clamp(sqrt(hp)/4.2, 0.55, 1.65)``
- ``unitLossRatio` ≈ R * casualtyScale`` (pak clamp na 0..1)
Speciálně: ``ram` + `caravan` mají ×1.15 (křehčí)`, ``knight` má ×0.82 (odolnější)`.

****3)** Jak se počítá “bojová síla” (power)** (``server/gameService.js:2577``)

Pro útok/obranu se počítá zvlášť:

- ``baseAttackPower` =  $\Sigma(\text{amount}(\text{unit}) * \text{attack}(\text{unit}) * \text{hpWeight}(\text{unit}))`$`
- ``baseDefensePower` =  $\Sigma(\text{amount}(\text{unit}) * \text{defense}(\text{unit}) * \text{hpWeight}(\text{unit}))`$`
- Jednotky s ``attack`≤0`` (v attack roli) nebo ``defense`≤0`` (v defense roli) se ****ignorují**** v sumě (tj. ram/caravan nepřidají power).

****4)** Taktický modifikátor složení armády** (``server/gameService.js:2639``)

Je to multiplikátor podle “mixu” jednotek, počítaný jen z ****combatantů**** (jednotky s `atk>0` nebo `def>0`; ram/caravan se nepočítají do share).

ÚTOK:

- cavalry share: ``+ min(10%, cavalryShare*12%)``
- scout share: ``+ min(5%, scoutShare*8%)``
- pokud je rytíř: ``×1.04``
- pokud militiaShare > 75%: ``×0.96``

OBRANA:

- archer share: ``+ min(8%, archerShare*10%)``
- pokud cavalryShare > 50% a archerShare < 10%: ``×0.95``

- pokud je rytíř: `x1.03`

Výsledný taktický multiplikátor je capnutý na `0.65..1.6`.

****5) Brána + opevnění – pravidla a multiplikátory****

Definice budov: `fortification` max 10 (`server/gameConfig.js:10`, `server/gameConfig.js:89`), `gate` max 1 (`server/gameConfig.js:11`, `server/gameConfig.js:99`).

****5.1 Fáze brány (damage + spotřeba beranidel)**** (`server/gameService.js:3087`)

- Pokud má obránce bránu (`gateLevel>0`) a útočník pošle beranidla:
 - `gateDamage = min(gateLevel, availableRams)`
 - brána se o to sníží a ****stejně množství beranidel se spotřebuje**** (`ramsConsumedOnGate = gateDamage`)
- => v praxi při `GATE\_MAX\_LEVEL=1` stačí 1 beranidlo na zničení brány.

****5.2 “Zastavení útoku” bránou + opevněním**** (`server/gameService.js:3122`)

Útok se ****neprovede**** a armáda se vrací, pokud platí:

- brána ****stále stojí**** (`gateStillStanding`)
- obránce má ****opevnění**** (`fortification>0`)
- útočník ****nemá**** beranidla (po odečtení těch spotřebovaných na bránu)

Pak:

- pokud obránce má posádku a lučištníky → útočník může mít “retreat” ztráty:
 - `archerPressure = log2(archers+1) \* 0.018`
 - `fortPressure = 0.01 + fortLevel\*0.006`
 - `retreatLossRatio = clamp(archerPressure + fortPressure, 0.015, 0.24)` (`server/gameService.js:3128`)
 - následně `applyCasualties(attacker, retreatLossRatio)`
- pokud obránce ****nemá posádku**** → retreatLossRatio = 0 (tj. žádné bojové ztráty).

****5.3 Pokud obránce nemá žádné jednotky**** (`server/gameService.js:3189` okolí)

- Boj se ukončí jako výhra útočníka a nejsou bojové ztráty (kromě toho, co se stalo na bráně – spotřebovaná beranidla už jsou pryč).

****5.4 Multiplikátory v samotném boji**** (`server/gameService.js:3248+`)

Po taktice:

- Pokud brána stále stojí: `defenseMultiplier \*= 1.08`
 - a pokud útočník má beranidla: `attackMultiplier \*= 1.04`
- Pokud existuje opevnění:
 - `defenseMultiplier \*= (1 + min(0.38, fortLevel\*0.028))`
 - + pokud obránce má lučištníky:
 - `defenseMultiplier \*= (1 + min(0.18, fortLevel\*0.018))`
- Pokud brána už ****nestojí**** a útočník má beranidla:
 - `attackMultiplier \*= 1.1` (RAM support) (`server/gameService.js:3269`)

- ale platí “musí přežít” viz bod 7.

****6) Rozhodnutí výsledku + model ztrát**** (`server/gameService.js:3280`, `server/gameService.js:2602`)

- `finalAttackPower = baseAttackPower \* attackMultiplier`
- `finalDefensePower = baseDefensePower \* defenseMultiplier`
- Útočník vyhrává, pokud `finalAttackPower > finalDefensePower` (`server/gameService.js:3317` okolí).

Ztráty se nedělají “unit vs unit”, ale přes 2 kroky:

1) Nejprve se spočítají ****loss ratio**** (kolik % armády má zemřít) podle poměru sil + health poolů (`resolveLossRatios`, `server/gameService.js:3280`):

- Používá se:
 - `attackShare`, `defenseShare` z celkové síly
 - “health pressure” = odmocnina poměru HP poolů (clamp 0.65..1.35)

2) Pak se loss ratio aplikuje na jednotlivé unit typy přes `applyCasualties` (`server/gameService.js:2602`):

- Každý unit typ má vlastní “odolnost” (viz `durability` a `casualtyScale×R` v tabulce).
- Ztráty jsou `Math.round(startAmount \* unitLossRatio)` → je to deterministické, ale u malých počtů může rounding dělat skoky.

****7) Speciální pravidlo: bonus beranidel jen když přežijí**** (`server/gameService.js:3325`)

Pokud se aplikoval RAM support (`attackMultiplier \*= 1.1`) a po aplikaci ztrát už útočník nemá žádné beranidlo:

- bonus se ****odebere****
- přepočítá se výsledek i loss ratios
- znovu se aplikuje `applyCasualties`

=> Beranidla jsou “support jednotka”: nedávají power přímo, ale drží multiplikátor, pokud přežijí.

****8) Zvěd (scout-only útok) je oddělená mechanika**** (`server/gameService.js:2708`, `server/gameService.js:2762`)

Pokud útočník pošle ****jen zvědy****:

- neproběhne `simulateAttackBattle`, ale spy logika
- obránce potřebuje `ceil(attackerScouts/2)` zvědů na úplné odražení (kill všech)
- jinak útočník ztratí `min(attacker, defender\*2)` zvědů
- kvalita získaných informací: `exact / approximate / none` podle toho, kolik zvědů přežilo (`server/gameService.js:2792+` v okolí `buildScoutIntelPayload`)

****9) Kořist (loot) a “balanced”** (`server/gameService.js:2837`, `server/gameService.js:2858`, `server/gameService.js:2921`)

- `lootPriority` se normalizuje; cokoliv neplatného spadne na `balanced` (`server/gameService.js:2837`).
- Kapacita kořisti = $\Sigma(\text{amount} * \text{lootCapacity}(\text{unit}))$ (`server/gameService.js:2849`)
- Loot se bere jen pokud `attackerWins` a útočník má přeživší (`server/gameService.js:4133`)

Distribuce:

- `balanced`: snaží se dělit kapacitu rovnoměrně mezi dostupné suroviny, když zbyde “zbytek”, rozdělí 1 po 1 v pořadí wood→stone→iron (`server/gameService.js:2858`)
- `wood/stone/iron`: bere maximum z vybrané suroviny, pak z dalších (`server/gameService.js:2921`)

****10) Dobývání osady rytířem + limit osad** (`server/gameService.js:4091`, `server/gameConfig.js:13`)

Dobytí (změna vlastníka) nastane, když:

- útočník vyhrál
- poslal rytíře a rytíř přežil
- obránce + všechny posily (support) jsou na 0
- cíl není vlastní osada
- a útočník nepřekročil limit osad pro daný svět/region (porovnává se `region`)
Limit je `MAX\_PLAYER\_VILLAGES = 6` (`server/gameConfig.js:13`).

Když dojde k dobytí, 1 rytíř se spotřebuje (`knightConsumed`) (`server/gameService.js:4154`).

****11) Supporty v obraně (posily)** (`server/gameService.js:2527`, `server/gameService.js:3898`, `server/gameService.js:4046`)

- Obránce do boje započítává i “stationed support” skupiny (posily) – pro výpočet síly se sloučí do jedné def selekce.
- Po boji se ale ztráty aplikují zvlášť:
 - posádka vesnice dostane `applyCasualties(..., battle.defenderLossRatio)` (`server/gameService.js:4046`)
 - každá support skupina dostane stejný `defenderLossRatio` samostatně.

Pokud chceš, můžu z toho udělat i “balancing report” (top jednotky podle atk/def per cost, per pop, vliv fort levelů na reálnou šanci vyhrát) a přidat pár konkrétních modelových bitev s ručním výpočtem podle vzorců.